

Integrantes

Guadalupe Sosa Acosta – 56604781
Matias Waller Scholderle – 55660867

1. Objetivos del práctico

Objetivo general:

Implementar un sistema funcional que simule la gestión de un kiosco, integrando clientes, vendedores, productos y compras, aplicando los principios de la orientación a objetos y respetando las restricciones legales y de negocio propias del dominio, de manera que las operaciones reflejen la realidad de un kiosco y mantengan la integridad de los datos.

Objetivos específicos:

- Analizar y modelar las relaciones entre las entidades del kiosco mediante UML, definiendo correctamente las clases, sus atributos, métodos y multiplicidades.
- Implementar el modelo UML en Python, traduciendo el diseño orientado a objetos en código fuente funcional y comentado.
- Aplicar los conceptos de abstracción, herencia, encapsulamiento y polimorfismo para organizar el sistema de manera eficiente y reutilizable.

2. Materiales

- Computadora con interpretador Python v3.9 o superior e IDE/editor de código-fuente;
- Librerías nativas de Python y la complementaria time.

3. Fundamentación teórica

3.1. Programación Orientada a Objetos (OOP)

La Programación Orientada a Objetos (OOP) es un paradigma que organiza el software en torno a objetos que combinan estado (atributos) y comportamiento (métodos), con el propósito de mejorar la modularidad, la reutilización y la mantenibilidad del código. Este enfoque permite diseñar componentes que se adaptan y se combinan sin rehacer el sistema entero, favoreciendo el desarrollo de sistemas complejos y escalables. (Morero, 2000)

La adopción de OOP aporta ventajas prácticas: reutilización de código mediante clases y bibliotecas, facilidad para extender y adaptar funcionalidad por herencia, protección de la implementación a través de la encapsulación y una estructura más modular que simplifica el mantenimiento y la evolución del software. Estas ventajas se reflejan tanto en el diseño conceptual como en la implementación concreta en lenguajes modernos. (Morero, 2000)

3.2. Clases, objetos y jerarquías (implementación y conceptos)

Una clase define la estructura y comportamiento compartido por un conjunto de objetos; una instancia (objeto) es la concreción de esa plantilla con valores propios. En la práctica, lenguajes como Python permiten expresar estas ideas de forma directa: las clases agrupan atributos y métodos, las instancias mantienen su estado independiente y las jerarquías de herencia favorecen la especialización y reutilización. Python además fomenta convenciones (por ejemplo, el uso de `self`, `__init__`, `__str__`) que facilitan la implementación de patrones orientados a objetos y el paso de funciones a métodos para hacer explícitas las relaciones entre tipos y operaciones. (Downey, 2012)

```
# Ejemplo sintáctico en Python (sin ejecutar)
class Tiempo:
    def __init__(self, hora=0, minuto=0, segundo=0):
        self.hora = hora
        self.minuto = minuto
        self.segundo = segundo

    def __str__(self):
        return f"{self.hora:02d}:{self.minuto:02d}:{self.segundo:02d}"
```

Fuente: elaboración propia

Las jerarquías se construyen mediante herencia: las clases superiores describen atributos y comportamientos generales, mientras que las subclasses añaden o redefinen detalles específicos (IS-A). Este mecanismo facilita la organización lógica del dominio y la reutilización de implementaciones cuando se pasan de la fase de diseño (diagramas) a la fase de código. (Downey, 2012; Morero, 2000)

3.3. Principios aplicables al diseño de clases

La encapsulación agrupa datos y métodos dentro de un objeto, controlando el acceso al estado interno mediante interfaces públicas; esto reduce el acoplamiento y facilita la modificación interna sin afectar a otras partes del sistema. La herencia permite extender comportamientos preexistentes, y el polimorfismo posibilita que distintos objetos respondan a un mismo mensaje con implementaciones propias, promoviendo flexibilidad y reducción de duplicación de código. En Python estos principios se aplican a través de convenciones de acceso, métodos especiales y la llamada dinámica a métodos, lo que facilita tanto el diseño como la refactorización del código. (Downey, 2012)

3.4. UML como herramienta de modelado para OOP

El Lenguaje Unificado de Modelado (UML) provee notaciones estandarizadas para representar la estructura y comportamiento de sistemas orientados a objetos. En particular, el diagrama de clases es la representación principal para mostrar nombres de clases, atributos, métodos y las relaciones entre clases; sirve como puente entre el modelado conceptual y la implementación en lenguajes como Python. UML facilita la comunicación del diseño, el control de dependencias y puede integrarse con herramientas que apoyan generación de código o verificación de modelos. (Rumbaugh et al., 1999; UML-diagrams.org, 2023)

3.5. Diagrama de clases: elementos y relaciones

Un diagrama de clases describe cada clase mediante nombre, atributos y métodos; además especifica las relaciones entre clases. Las relaciones principales son: asociación (conexión general entre instancias), agregación (relación parte-todo débil donde las partes pueden existir independientemente del todo), composición (relación parte-todo fuerte donde las partes dependen de la existencia del todo) e herencia (generalización que modela relaciones IS-A). Estas distinciones ayudan a diseñar cómo se gestionará la creación, la dependencia y la vida de los objetos en la implementación posterior. (Rumbaugh et al., 1999; UML-diagrams.org, 2023)

3.6. Agregación, composición y multiplicidad (notación y significado)

La **agregación** modela que un objeto “tiene” referencias a otros objetos que pueden sobrevivir si el contenedor se elimina (por ejemplo, un Departamento tiene Empleados que existen fuera del Departamento). La **composición** expresa una relación más fuerte en la que las partes tienen su ciclo de vida controlado por el todo (por ejemplo, un Coche compone Motor y Ruedas: si el Coche deja de existir, las partes no tienen sentido por separado). La **multiplicidad** indica cuántas instancias participan en la relación (p. ej. 1, 0..1, 1..*, 0..*), y se anota junto a las líneas en el diagrama para aclarar restricciones de cantidad. Estas convenciones y su notación están descritas en la literatura de modelado y en guías pedagógicas sobre UML. (UML-diagrams.org, 2023; Lucid Software, 2023)

Un ejemplo conceptual que suele usarse al modelar sistemas es: un **Cliente** puede tener 0..* **Órdenes** (multiplicidad 0..*); un **Departamento** puede agregar **Empleados** (agregación); y un **Coche** puede contener como parte una instancia de **Motor** creada en su interior (composición). Estas elecciones de modelo guían decisiones de implementación (por ejemplo, quién crea y destruye objetos) y de persistencia. (UML-diagrams.org, 2023; Lucid Software, 2023)

3.7. Integración práctica: de UML a Python

El modelado con UML y la implementación en Python son complementarios: UML ayuda a definir la estructura (clases, relaciones, multiplicidades y visibilidades) y Python permite implementarla con clases, métodos y convenciones de acceso. Al pasar del diagrama al código conviene traducir explícitamente los contratos (métodos públicos), respetar las decisiones de composición vs. agregación en la gestión de creación/destrucción de objetos y documentar las multiplicidades mediante validaciones/colecciones en tiempo de ejecución. En resumen, diseñar primero en UML y luego implementar en Python facilita coherencia, trazabilidad y mantenimiento del sistema. (Rumbaugh et al., 1999; Downey, 2012)

4. Metodología

Para la realización del laboratorio, se comenzó interpretando cuidadosamente la consigna, la cual establecía que el sistema debía modelar un kiosco utilizando seis clases y aplicando los cuatro principios de la orientación a objetos. Como primer paso, se analizó el diagrama UML proporcionado en el Apéndice A y, sobre esa base, se elaboró un UML propio (Apéndice B) que representara de manera precisa la estructura necesaria para gestionar clientes, vendedores, productos y compras.

Se definió la clase **Fecha** como un componente central para controlar vencimientos de productos y registrar fechas de nacimiento e ingreso de personas. Esta clase permitió implementar validaciones que se reflejan en los resultados, como impedir el ingreso de funcionarios menores de 15 años, evitar registros o compras en días no laborables y garantizar que la fecha de vencimiento de los productos sea posterior a la fecha actual, asegurando consistencia en los datos y evitando operaciones inválidas.

La clase **Persona** se estableció como base para concentrar atributos comunes a clientes y vendedores, como nombre, cédula y teléfono. La herencia permitió crear las subclases **Cliente** y **Vendedor**, incorporando atributos y métodos específicos, tales como **saldoCuenta** para clientes y **sueldo** y **turno** para vendedores. El método **mostrarPersona()** se redefinió en cada subclase, aplicando polimorfismo para mostrar información diferenciada según el tipo de persona, respetando la lógica del kiosco. Esta decisión de diseño facilitó la implementación de reglas de negocio, como restricciones de edad mínima y control de categorías de productos según la edad.

La clase **Producto** se diseñó para representar los artículos del kiosco, incluyendo atributos como categoría, stock, unidad y precio, y métodos para agregar o reducir stock y modificar precios. Las categorías predefinidas (alimentos, bebidas, tabaco y alcohol, lotería, higiene y cuidado personal, papelería y electrónica) fueron seleccionadas para abarcar la totalidad de productos típicos de un kiosco. Además, se implementó la eliminación automática de productos vencidos al inicio del programa, asegurando que el inventario se mantuviera consistente con las operaciones reales.

La clase **Compra** se definió como núcleo del sistema de ventas, asociándose con **Cliente**, **Vendedor**, **Producto** y **Fecha**. Se implementaron reglas de negocio como la obligatoriedad de incluir al menos un

producto por compra, la distinción entre pagos en efectivo y con tarjeta, y la generación automática de facturas tipo C con numeración consecutiva y número de local fijo, cumpliendo la normativa uruguaya para pequeños contribuyentes. Esto evitó inconsistencias como compras vacías o números de factura duplicados.

En cuanto a las relaciones, se estableció composición entre **Persona** y **Fecha**, indicando que las fechas dependen de la existencia de la persona; agregaciones entre **Fecha** y **Producto** o **Compra**, **Cliente** y **Compra**, **Vendedor** y **Compra**, y **Producto** y **Compra**, mostrando asociaciones sin dependencia de vida; y herencia de **Persona** hacia **Cliente** y **Vendedor** para reutilizar atributos y métodos comunes. La asignación de multiplicidades permitió reflejar correctamente la cantidad de objetos implicados, como que un cliente puede realizar múltiples compras, cada compra tiene al menos un producto, y una misma fecha puede aplicarse a varios productos o compras.

La implementación en Python (Apéndice C) respetó fielmente el UML y estas decisiones de diseño. Se incorporaron validaciones en tiempo real para: cédulas, nombres, edad mínima y máxima, días no laborables, categorías de productos, unidades y stock, duplicación de códigos y teléfonos, y modalidad de pago según la lógica del kiosco. Los atributos críticos se encapsularon mediante atributos privados y getters/setters, asegurando que cualquier modificación cumpla las reglas definidas y evitando accesos indebidos. Se realizaron pruebas con archivos de ejemplo (Apéndice E) que confirmaron el correcto funcionamiento de todas las validaciones y restricciones, asegurando que los flujos de ingreso, compra y manejo de inventario se comportaran según lo planificado.

Durante todo el desarrollo se realizaron pruebas modulares para verificar la funcionalidad de cada clase y método. Posteriormente, se integraron los módulos y se validó el sistema completo frente a los requisitos del kiosco (Apéndice D), comprobando la coherencia de operaciones, reglas de negocio y restricciones implementadas. Esta rutina de verificación y validación permitió detectar y corregir errores de forma temprana, asegurando la integridad de los datos y el correcto funcionamiento del sistema.

5. Resultados

5.1. Resultados del UML

El UML desarrollado (Apéndice B) refleja de manera conceptual la estructura y funcionamiento de un kiosco real, permitiendo visualizar las clases, relaciones, atributos, métodos y restricciones de manera clara. Se diseñaron varias clases principales: **Fecha**, **Persona**, **Cliente**, **Vendedor**, **Producto** y **Compra**, así como las relaciones entre ellas.

La **clase Fecha** se definió con atributos de día, mes y año, y métodos para calcular diferencias entre fechas, asignar nuevas fechas y mostrar la fecha en formato legible. Su importancia radica en centralizar toda la lógica temporal del sistema, incluyendo edad de personas, fechas de ingreso de personal y vencimiento de productos. En el UML se representa mediante **composición con Persona**, lo que significa que la existencia de la fecha depende completamente de la existencia de la persona; sin una persona, no tiene sentido que exista una fecha de nacimiento o de ingreso. Además, **Fecha** mantiene una **agregación con Producto y Compra**, reflejando que estos objetos pueden existir independientemente, pero requieren información temporal para validar operaciones, como la caducidad de lotes o la fecha de transacción de una compra.

La **clase Persona** se diseñó como base para la herencia hacia **Cliente** y **Vendedor**. Incluye atributos como código único, cédula de identidad, nombre, fechas de nacimiento e ingreso, y teléfono. Entre sus métodos se incluyen getters y setters, y el método `mostrarPersona()`, que permite mostrar la información de la persona de manera estandarizada. Los atributos críticos, como `-codigo`, se declararon privados y solo se accede a ellos mediante métodos `setCodigo()` y `getCodigo()`, implementando encapsulamiento para proteger la integridad de los datos y controlar las modificaciones. En el UML se indica que **Cliente** y **Vendedor** heredan de **Persona**, lo que facilita la reutilización de atributos y métodos comunes, evitando duplicación de código y permitiendo el polimorfismo: cada subclase re-define `mostrarPersona()` para mostrar información específica, como saldo de cuenta o historial de ventas.

La **clase Vendedor**, que hereda de **Persona**, agrega atributos como sueldo y turno, y métodos para registrar ventas, mostrar información y dar de baja al vendedor. Su relación de **agregación con Compra** indica que un vendedor puede existir sin haber realizado ventas, pero que cada venta registrada se asocia a un vendedor. Esto permite controlar la trazabilidad de las transacciones sin limitar la existencia independiente de los vendedores. El encapsulamiento también se aplica a atributos sensibles como `-sueldo`, que solo puede modificarse mediante métodos definidos.

La **clase Cliente**, también heredada de **Persona**, incorpora un atributo `saldoCuenta`, que permite simular sistemas de efectivo o tarjeta. Sus métodos incluyen `sumarSaldo()`, `pagarCompra()`, `mostrarPersona()` y `darDeBajaCliente()`. La agregación con **Compra** refleja que un cliente puede existir sin compras, pero que cada compra realizada queda asociada a él. En el UML también se refleja la multiplicidad: un cliente puede tener múltiples compras (`0..*`), y cada compra debe tener al menos un producto (`1..*`). El atributo `saldoCuenta` está protegido mediante encapsulamiento, de modo que solo se puede modificar a través de los métodos de la clase que aplican las validaciones correspondientes.

La **clase Producto** incluye atributos como código, nombre, categoría, cantidad disponible, unidad de medida, precio y fecha de vencimiento. Sus métodos permiten gestionar el stock, modificar el precio, obtener la fecha de vencimiento y mostrar información del producto. Los atributos privados, como `-codigo` o `-cantidad`, están protegidos mediante encapsulamiento y se accede a ellos únicamente mediante métodos específicos, garantizando la consistencia de los datos. En UML, la agregación con **Compra** refleja que un producto puede estar en cero o más compras, y la agregación con **Fecha** asegura que cada lote de producto tenga su fecha de vencimiento registrada sin depender estrictamente del objeto fecha, manteniendo independencia conceptual.

La **clase Compra** incluye atributos como código, número de factura, fecha de compra, modalidad de pago, lista de productos, estado y total. Sus métodos permiten agregar productos, calcular el total, anular la compra y mostrar la información completa. Los atributos críticos, como `-codigo` y `-estado`, están encapsulados, de forma que solo puedan modificarse mediante métodos que implementan validaciones, evitando inconsistencias o manipulaciones indebidas. La agregación con **Cliente**, **Vendedor**, **Producto** y **Fecha** refleja que estos objetos participan en la transacción, pero pueden existir de manera independiente. Las multiplicidades y relaciones reflejadas en UML aseguran coherencia con la lógica de negocio: cada compra debe tener al menos un producto, y los productos pueden estar asociados a múltiples compras.

El UML proporciona una **visión conceptual clara del sistema**, mostrando herencia, composición, agregación, multiplicidad y encapsulamiento, dejando explícitas las dependencias y relaciones entre objetos que después se implementan en código.

5.2. Resultados del Código

El código implementa fielmente las decisiones tomadas en el UML, añadiendo la lógica de negocio, validaciones y restricciones necesarias para que el sistema funcione como un kiosco real.

Gestión de fechas: La clase **Fecha** permite calcular edades, validar fechas de ingreso, nacimiento y vencimiento de productos. Por ejemplo, al registrar un funcionario, el código verifica automáticamente que la fecha de ingreso sea posterior a la fecha en que cumple 15 años, deteniendo el registro si no se cumple. Además, se valida que la fecha de vencimiento de los productos sea posterior a la fecha actual, y que las compras solo se registren en días hábiles. Esto asegura que los datos sean consistentes con la realidad del kiosco y evita errores de operación fuera de horario o con productos vencidos.

Validaciones de personas: Al crear clientes o vendedores, el código valida que las cédulas cumplan el formato uruguayo y que no se repitan. Los nombres deben contener únicamente letras. Se calcula automáticamente la edad a partir de la fecha de nacimiento y la fecha actual, deteniendo el registro si no se cumple la edad mínima o si supera la edad máxima de 120 años. Para clientes menores, el sistema impide la compra de tabaco, alcohol o lotería, y restringe la compra de lotería en efectivo. Para vendedores menores de edad, también se bloquea la venta de productos de estas categorías.

Además, el **encapsulamiento** se aplica mediante la declaración de atributos privados en las clases **Persona**, **Cliente** y **Vendedor**, como `-codigo` o `-saldoCuenta`. Estos atributos solo pueden ser modificados a través de getters y setters que incluyen validaciones, protegiendo la integridad de los datos

y evitando accesos directos indebidos. Esto asegura que las restricciones de edad, cédula, nombre y categoría se apliquen siempre de forma consistente.

Gestión de productos: Cada producto tiene un código único, nombre válido, categoría definida y stock controlado. Se implementan métodos para agregar o reducir stock, modificar precios y mostrar información. Al iniciar el sistema, se eliminan automáticamente los productos vencidos, manteniendo solo lotes vigentes. Las categorías predefinidas (alimentos, bebidas, tabaco y alcohol, lotería, higiene y cuidado personal, papelería, electrónica) reflejan la lógica de negocio de un kiosco y están alineadas con los atributos del UML. Los atributos sensibles, como `-cantidad` o `-codigo`, también están encapsulados, garantizando que solo puedan modificarse mediante métodos que validan la información.

Compras y facturación: Cada compra contiene al menos un producto, cumpliendo la multiplicidad indicada en UML. Se implementan modalidades de pago: efectivo, tarjeta y fiado. En pagos con tarjeta, el saldo del cliente se descuenta automáticamente mediante el método `pagarCompra()`. La numeración de facturas tipo C se genera de forma consecutiva, incluyendo un número fijo de local y un número incremental de compra. Esto evita duplicaciones y asegura trazabilidad y cumplimiento de la normativa fiscal uruguaya. Cada compra queda asociada al cliente y al vendedor mediante referencias, reflejando la agregación definida en UML. Los atributos críticos de `Compra`, como `-codigo` o `-estado`, están encapsulados para que solo puedan modificarse mediante métodos que implementan las validaciones correspondientes.

Control de duplicaciones y consistencia: Antes de registrar cualquier cliente, vendedor o producto, el sistema verifica que no existan códigos o cédulas duplicadas. Al cambiar de rol entre cliente y funcionario, se conservan los datos relevantes y se respetan las restricciones de edad y categoría. Estas validaciones detienen el flujo de ingreso en tiempo real ante errores, evitando datos inconsistentes y bucles infinitos.

Polimorfismo y reutilización de métodos: El método `mostrarPersona()` está definido en `Persona` y se redefine en `Cliente` y `Vendedor`. Esto permite mostrar información específica según el tipo de persona: saldo y compras para clientes, sueldo y ventas para vendedores. Esta implementación refleja directamente la herencia y polimorfismo definidos en UML, demostrando reutilización de código y adaptabilidad.

Integración entre objetos: Las relaciones de agregación entre clientes, vendedores, productos, fechas y compras permiten que los objetos existan de manera independiente pero se vinculen temporalmente para operaciones específicas. Por ejemplo, un producto puede estar en múltiples compras o ninguna, un cliente puede existir sin compras y un vendedor puede existir sin ventas registradas. Estas relaciones permiten modelar de manera realista las interacciones dentro de un kiosco, tal como se indica en UML.

Evidencia mediante archivos de prueba: Los archivos incluidos en el Apéndice E muestran ejemplos completos de creación de clientes, vendedores, productos y compras. Permiten verificar que todas las validaciones y restricciones funcionan correctamente: control de edad, fechas de ingreso y vencimiento, categorías de productos, modalidades de pago, eliminación de productos vencidos, y prevención de duplicaciones. Estos archivos prueban que la lógica del sistema cumple con el comportamiento esperado y que el código implementa de manera fiel la estructura conceptual del UML.

Finalmente, los resultados muestran que el UML proporciona la visión conceptual del sistema, incluyendo herencia, composición, agregación, multiplicidad, relaciones entre objetos y encapsulamiento, mientras que el código implementa todas estas decisiones de forma operativa, añadiendo validaciones, restricciones y lógica de negocio que permiten que el sistema funcione como un kiosco real. La correspondencia entre UML y código es clara: cada clase, relación y restricción conceptual tiene su implementación concreta, asegurando consistencia, trazabilidad y fidelidad a la lógica de negocio definida.

6. Conclusiones

A partir del desarrollo del sistema, se logró implementar un modelo funcional de un kiosco que integra la gestión de clientes, vendedores, productos y compras, respetando los principios de la orientación a objetos y las restricciones propias del dominio. El sistema permite controlar de manera coherente las edades mínimas y máximas de las personas, la validez de las fechas de nacimiento, ingreso y vencimiento de productos, así como las operaciones de compra según modalidades de pago y categorías de productos, cumpliendo con la normativa uruguaya vigente.

Entre los objetivos alcanzados se puede destacar la correcta gestión de inventario, asegurando que los productos vencidos se eliminen automáticamente y que las compras contengan siempre al menos un producto; la aplicación de reglas de negocio sobre pagos en efectivo o tarjeta; la validación de cédulas, nombres y códigos para evitar duplicaciones; y la implementación de facturación consecutiva tipo C, garantizando trazabilidad y consistencia fiscal.

Durante el proceso se corrigieron errores relevantes, como el uso del valor "0" para volver en los menús sin registrarlo como entrada, y la preservación de lotes de productos para mantener las fechas de vencimiento correctas, evitando pérdida de información. Estas mejoras optimizaron la robustez del sistema y aseguraron que las operaciones reflejaran de manera más fiel la lógica de funcionamiento de un kiosco real.

En conjunto, el desarrollo cumplió con los objetivos de crear un sistema coherente, seguro y consistente, que respeta las reglas de negocio y la normativa vigente, incorporando mecanismos de validación y control que evitan inconsistencias y errores comunes en la gestión de información.

Referencias

- [1] Morero, F., *Introducción a la OOP*, versión 1.0.0, Montevideo, Grupo EIDOS, 2000.
- [2] Rumbaugh, J.; Jacobson, I.; Booch, G., *The Unified Modeling Language Reference Manual*, Addison-Wesley, 1999. ISBN 0-201-30998-X.
- [3] UML-diagrams.org, *UML Class and Object Diagrams Overview*, <https://www.uml-diagrams.org/class-diagrams-overview.html>, 2023.
- [4] Downey, A., *Think Python: How to Think Like a Computer Scientist*, versión 2.0.17, Green Tea Press, 2012.

A. Diagrama UML a considerar.

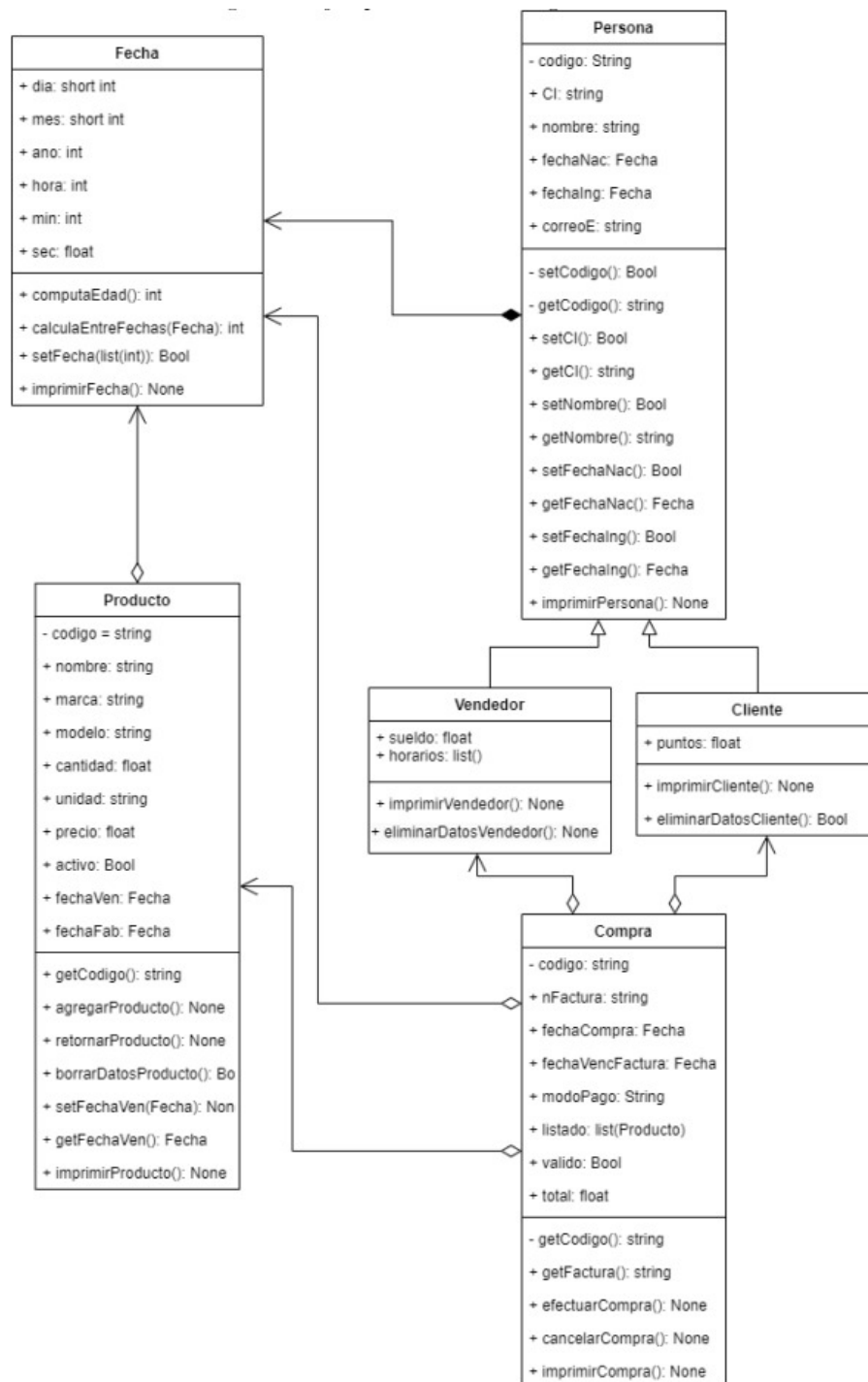


Figura 1: Fuente: Elaboración propia

B. Diagrama UML Implementado

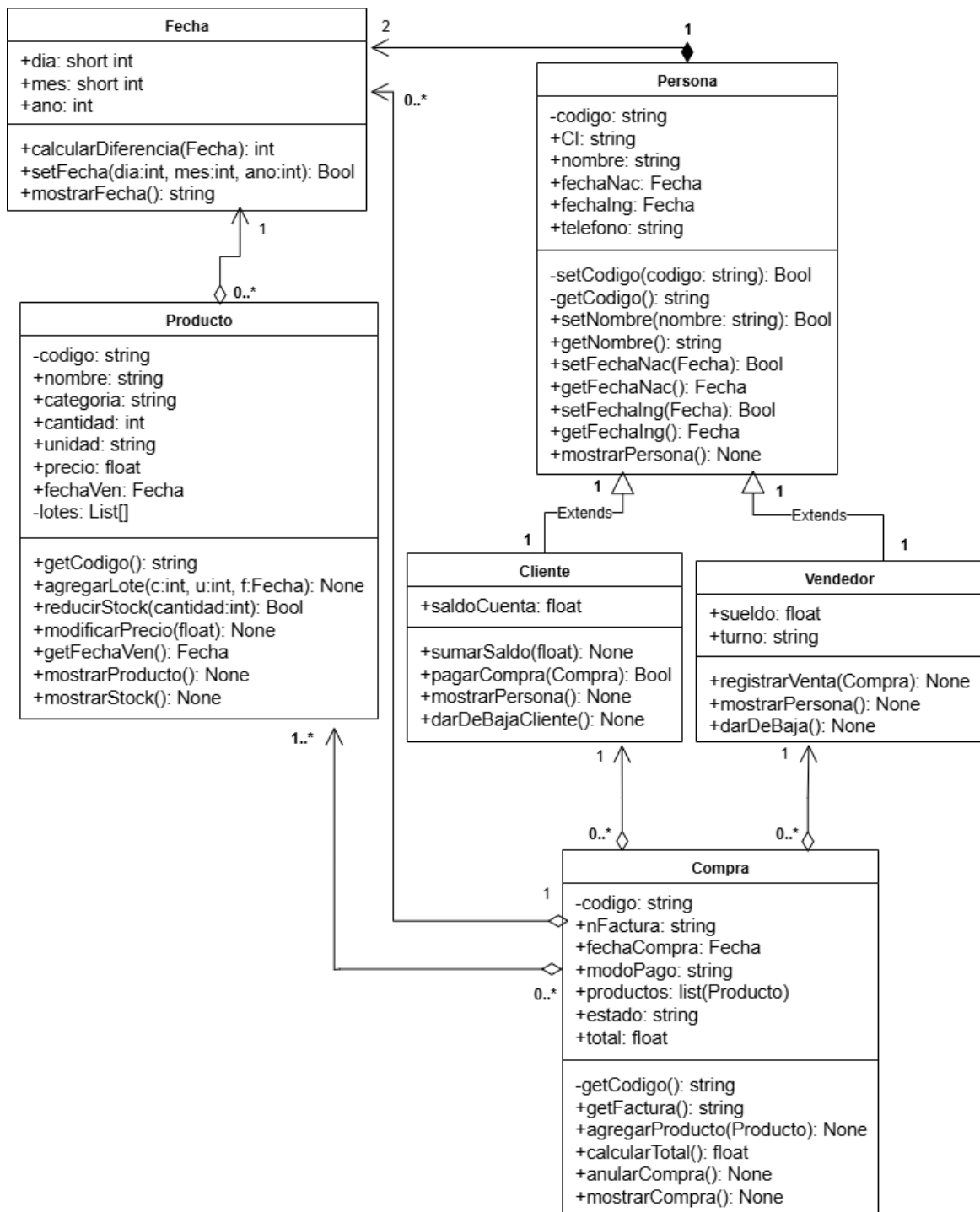


Figura 2: Fuente: Elaboración propia

C. Código fuente en Python

La implementación del sistema en Python se encuentra documentada y comentada en el siguiente repositorio, así como los archivos implementados:

https://colab.research.google.com/drive/1uzlicgZobbOjRXET8VFxqgy_sUXCXUKG#scrollTo=MP187-Vb2n5O

Fuente: Elaboración propia

D. Pruebas del sistema.

D.1. Prueba 1: Funcionamiento estandar

<https://docs.google.com/document/d/16JfsKpRvA2tc2aZaeRkkyKuhrYEpd08XgRLGrIa7H7E/edit?usp=sharing>

Fuente: Elaboración propia

D.2. Prueba 2: Prueba de errores/valores invalidos

<https://docs.google.com/document/d/104XJd2NJdd4QicNo08v8J7chsd0mXnPqxZXcQk0iV04/edit?usp=sharing>

Fuente: Elaboración propia

D.3. Prueba 3: Prueba de cancelaciones

<https://docs.google.com/document/d/1TIFtJgzpC4sAIG6rKH054x9AXH7Q7l9eMxlEjvz0Awk/edit?usp=sharing>

Fuente: Elaboración propia

E. Archivos utilizados.

E.1. Archivo clientes.txt”.

```
1 B7G8H;4.321.987-0;JUAN PEREZ;15;5;2010;15;5;2025;+598 2345 6789;1260.0
2 K2L9M;1.234.567-8;MARIA GOMEZ;20;3;2005;20;3;2025;+598 3456 7890;3000.0
3 T6R3X;7.654.321-2;CARLOS DIAZ;5;7;2012;5;7;2025;+598 4567 8901;1200.0
4 P9Q2Z;3.210.987-6;SOFIA MARTINEZ;12;11;2008;12;11;2025;+598 5678 9012;1800.0
5 V1W4Y;5.678.901-3;LUIS HERNANDEZ;8;2;2015;8;2;2025;+598 6789 0123;1000.0
6 R5S8T;9.876.543-4;ISABEL FERNANDEZ;17;9;2007;17;9;2025;+598 7890 1234;2265.0
7 H2K5P;8.210.345-7;FEDERICO CASTRO;14;1;2011;14;1;2025;+598 0123 4567;1700.0
8 JSDKC;5.555.444-8;SOFIA RAMIREZ;1;12;2020;2;10;2025;+598 1111 0202;10.0
9 S5FSD4;5.666.545-8;VICTORIA ETCHEVERRY;24;5;2005;8;11;2024;+598 2548 6598;550.0
10 CD4S5S;1.325.624-8;PABLO CASTILLO;2;8;2002;5;12;2023;+598 4785 6598;4554.0
11 56DSVS;1.963.852-7;ANAMARIA ACOSTA;8;8;2008;12;12;2024;+598 4747 5252;789.0
```

Figura 3: Fuente: Elaboración propia

E.2. Archivo compras.txt”.

```
1 B7G8H;C-0105-0001;2;10;2025;tarjeta;completada;240.0;FR65SD:3
2 R5S8T;C-0105-0002;2;10;2025;tarjeta;completada;235.0;S1T2U3:1,FR65SD:2
3 B7G8H;C-0105-0003;2;10;2025;efectivo;completada;400.0;J1K2L3:5
4 V1W4Y;C-0105-0004;3;10;2025;efectivo;completada;370.0;VCS564:1,FR65SD:4
```

Figura 4: Fuente: Elaboración propia

E.3. Archivo ”productos.txt”.

```
1 D4E5F6;Jugo de Naranja;bebidas;80.0;
2 G7H8I9;Cerveza;tabaco y alcohol;150.0;
3 J1K2L3;Raspadita;loteria;80.0;40/1/2/10/2070
4 M4N5O6;Papel A4;papeleria;250.0;48/100/1/12/2050,50/100/12/5/2060
5 S1T2U3;Auriculares;electronica;75.0;72/41/2/10/2025
6 V4W5X6;Chocolate;alimentos;90.0;
7 Y7Z8A9;Agua Mineral;bebidas;40.0;
8 B1C2D3;Cigarrillos;tabaco y alcohol;120.0;
9 FR65SD;Jugo de Manzana;bebidas;80.0;90/1/5/10/2025
10 VCS564;Quiniela;loteria;50.0;499/1/4/10/2025
11 E5V3S5;Laptop;electronica;1000.0;200/1/30/11/2030
```

Figura 5: Fuente: Elaboración propia

E.4. Archivo "vendedores.txt".

```
1 FA5SD3;5.895.624-8;KAMILA IZQUIERDO;2;10;2005;2;10;2025;+598 1111 2222;30000.0;tarde
2 GB7HJ9;7.123.456-0;MARIO PEREZ;15;5;2008;15;5;2025;+598 2222 3333;25000.0;mañana
3 HJ8KL1;3.987.654-2;LUISA RAMOS;30;8;2000;30;8;2025;+598 3333 4444;40000.0;mañana
4 KL2MN4;2.456.789-3;DIEGO FERNANDEZ;12;12;2010;12;12;2025;+598 4444 5555;20000.0;tarde
5 OP7QR8;1.234.567-8;NICO GOMEZ;5;6;2012;5;6;2025;+598 6666 7777;18000.0;noche
6 ST1UV2;9.876.543-0;SOFIA LOPEZ;18;11;2007;18;11;2025;+598 8888 9999;30000.0;tarde
7 UV3WX4;4.321.098-7;JULIAN CASTRO;8;9;2011;8;9;2025;+598 9999 0000;22000.0;noche
8 WX5YZ6;0.987.654-3;ISABELLA TORRES;28;2;1998;28;2;2025;+598 1010 1111;50000.0;noche
9 VD5SS4;5.555.655-7;SOFIA MODERNELL;6;6;2003;5;7;2022;+598 4785 6523;23605.0;mañana
10 56DSC4;2.365.246-8;DANIEL MARTINEZ;4;12;2001;7;11;2024;+598 1111 5485;23700.0;noche
11 WE48C6;6.636.252-8;MARIA CASTILLO;7;9;1998;4;6;2019;+598 1245 6585;24000.0;mañana
```

Figura 6: Fuente: Elaboración propia